Họ và tên: Trương Chí Long
MSSV / Lớp: 3123410198 / DCT1234

BÀI TẬP TÌM KIẾM TRONG AI

Bài 1:

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, F, B, G
Path	S → D → G
Total Cost	10 (S → D: 2, D → G: 8)

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E, G
Path	S → C → F → E → G
Total Cost	5 (S → C: 2, C → F: 1, F → E: 0, E → G: 2)

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G (Tìm thấy đích G)

Path cuối cùng: S → D → G
Total Cost: 10

Câu hỏi thảo luận:

Tiêu chí	**BFS**	**DFS**
Frontier	Hàng đợi FIFO	Ngăn xếp LIFO hoặc đệ quy
Tối ưu	Tối ưu theo số cạnh nếu mỗi bước có cùng chi phí	Không đảm bảo tối ưu

Bài 2:

2.1. Biểu diễn trạng thái

s = (p_1, p_2, …, $p_□$), với $p_i \in \{1,2,3\}$ là cọc chứa đĩa i.

2.2. Kích thước không gian trạng thái
$|S| = 3^n$
Mỗi đĩa chọn độc lập 1 trong 3 cọc, nên có 3^n gán. Mọi gán đều tương ứng đúng một cấu hình hợp lệ, vì trên mỗi cọc các đĩa chỉ có một cách xếp (lớn dưới, nhỏ trên).

2.3. Trạng thái bắt đầu
$S_0 = (1, 1,..., 1)$ (Tất cả n đĩa đều đang ở cọc 1).

2.4. Các hành động hợp lệ
Actions(s) = {move(i, a→b)}
Điều kiện:

1. a = p_i (đĩa i đang ở cọc a) và b ≠ a.
2. Đĩa i đang ở đỉnh cọc a: $\forall$ j < i, $p_□ \neq$ a.
3. Đặt được lên cọc b: $\forall$ j < i, $p_□ \neq$ b (cọc b rỗng hoặc đỉnh b lớn hơn i).

2.5. Goal Test
goalTest(s) = True khi và chỉ khi s = (3, 3, …, 3), tức $\forall$ i: $p_i = 3$.

Bài 3:
3.1. Thiết kế heuristic admissible và consistent

Node	**h(n)**
S	8
A	10
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế
Explored Set: S, B, D, G.
Path: S → B → D → G.
Total Cost: 8.

3.3. Heuristic admissible nhưng không consistent
Đề xuất giá trị: $h(G) = 0, h(D) = 1, h(C) = 6$

Cạnh vi phạm: Cạnh $C \to D$ có cost $c(C, D) = 2$.

3.4. Câu hỏi mở rộng
Lấy đường tối ưu n = n_0, n_1, …, n_k = G.
Áp dụng consistent dọc đường đi:
$h(n_0) \le c(n_0,n_1) + h(n_1) \le c(n_0,n_1) + c(n_1,n_2) + h(n_2) \le \ldots \le \Sigma c(n_i,n_{i+1}) + h(G) = h^*(n) + 0$.
Vậy $h(n) \le h^*(n)$, tức h admissible.

Bài 4:
4.1. Depth-First Search
Explored Set: A, C, F, B, G.
Path: A → C → G.
Cost: 3.

4.2. Breadth-First Search
Explored Set: A, C, D, E.
Path: A → E.
Cost: 3.

4.3. Uniform-Cost Search
Explored Set: A, D, C, G.
Path: A → D → G.
Cost: 2.

4.4. Greedy Search
Explored Set: A, E.
Path: A → E.
Cost: 3.

4.5. A* Search
Explored Set: A, D, G.
Path: A → D → G.
Cost: 2.

Bảng tổng hợp

Thuật toán	**Tiêu chí chọn node**	**Path**	**Cost**
DFS	Độ sâu lớn nhất	A-C-G	3
BFS	Độ sâu nhỏ nhất	A-E	3
UCS	g(n)	A-D-G	2
Greedy	h(n)	A-E	3

A*	g(n)+h(n)	A-D-G	2

Bài 5:
Câu 1: SAI. DFS có thể đi thẳng tới đích và mở ít nút hơn A*.

Câu 2: SAI. Quân xe đi được nhiều ô trong một nước nên Manhattan distance có thể ước lượng thừa.

Câu 3: ĐÚNG. Pacman đi theo 4 hướng, mỗi bước chi phí 1. Mọi đường đi thực không ngắn hơn đoạn thẳng nối hai điểm, nên h_Euclid ≤ h*.

Câu 4: SAI. Với $h_1 = h_2 = h^*$ (admissible), tổng là $2h^* > h^*$ nếu $h^* > 0$, vượt chi phí thực.

Câu 5: SAI. Consistent ⇒ admissible nhưng không có chiều ngược lại.

Câu 6: SAI. A* graph search chỉ đảm bảo tối ưu khi heuristic consistent (hoặc có mở lại nút đã đóng).

BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Câu 1:

a) Chuỗi hành động ngắn nhất

Rẽ phải (hướng Đông, v=0)
Tăng tốc (đạt v=1, di chuyển đến 1,0)
Giảm tốc (đạt v=0, dừng tại 1,0)
Rẽ trái (hướng Bắc, v=0)
Tăng tốc (đạt v=1, di chuyển đến 1,1)
Giữ tốc độ (v=1, di chuyển đến 1,2)
Giữ tốc độ (v=1, di chuyển đến 1,3)
Giảm tốc (đạt v=0, dừng tại 1,3)
Rẽ trái (hướng Tây, v=0)
Tăng tốc (đạt v=1, di chuyển đến 0,3)
Giảm tốc (đạt v=0, dừng tại 0,3)
Rẽ phải (hướng Bắc, v=0)
Tăng tốc (đạt v=1, di chuyển đến 0,4)
Giảm tốc (đạt v=0, dừng tại 0,4)
Rẽ trái (hướng Tây, v=0)

b) Kích thước không gian trạng thái

$|S| = 4 \cdot M \cdot N \cdot (Vmax + 1)$

c) Có. Ở v = 0 luôn có thể rẽ. Ở v = 1 luôn có thể giảm tốc xuống 0 mà không di chuyển. Nhưng ở $v \geq 2$ mà tường ở ngay phía trước thì cả ba lệnh đều va tường, và không được rẽ vì $v \neq 0$.

d) Hệ số phân nhánh tối đa b = 3.
Khi v = 0: {Faster, Left, Right} (không có Maintain và Slower).
Khi 0 < v < Vmax: {Faster, Maintain, Slower}.
Khi v = Vmax: chỉ có 2 hành động.

Ví dụ: ((1,0), Bắc, v=0) có Faster (lên (1,1)), Rẽ trái, Rẽ phải. Cả ba đều hợp lệ.

e) Không. Một hành động có thể đi nhiều ô (tới Vmax ô), nên Manhattan distance có thể lớn hơn số hành động thực tế.

Phản ví dụ trên một hành lang thẳng dài 6 ô, $Vmax \geq 2$, xe đã đúng hướng với v=0:
Số hành động tối ưu: Faster (đi 1), Faster (đi 2), Maintain (đi 2), Slower (đi 1), Slower (đi 0) = 5 hành động.
$h = Manhattan = 6 > h^* = 5$.

Với lưới 5×5 đã cho, hành lang thẳng dài tối đa 4 ô nên chưa vi phạm. Nhưng admissibility phải đúng cho mọi trường hợp, nên heuristic này không admissible.

f) Có. Mỗi hành động làm vị trí thay đổi tối đa $v' \leq V_{max}$ ô theo một trục, nên Manhattan distance giảm tối đa V_{max} mỗi hành động. Các hành động rẽ không di chuyển. Do đó số hành động tối thiểu ≥ Manhattan/V_{max}. Vậy h = Manhattan/$V_{max} \leq h^*$, tức admissible.

g) Không (tính đầy đủ không đổi). Đồ thị hữu hạn nên h bị chặn bởi một hằng H. Mọi nhánh vô hạn có $g \to \infty$ (mỗi hành động chi phí ≥ 1), nên $f = g + h \to \infty$. Nút đích có f hữu hạn, và chỉ có hữu hạn nút với f nhỏ hơn giá trị đó. Vì vậy cuối cùng đích được mở.

h) Có. Heuristic không admissible có thể đánh giá thừa một nút trên đường tối ưu. Khi đó A* mở nút đích qua đường kém hơn trước, nên trả về đường không tối ưu.

i)
Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.
Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

Câu 2:
a) BFS: $\{ n : d(n) < d(n_s) \}$.
BFS mở hết các tầng nông hơn đích nông nhất.

b) UCS: $\{ n : g(n) < g(n_c) \}$.
UCS mở mọi nút rẻ hơn đường đích tối ưu.

c) A* (h nhất quán): $\{ n : g(n) + h(n) < g(n_c) \}$.
Với h consistent, f không giảm dọc đường, nên mọi nút có $f < C^*$ chắc chắn được mở (các nút $f = C^*$ tùy tie).

Câu 3:
a)

Thuật toán	**A-C-E-G**	**A-C-E-F-G**	**A-B-D-E-F-G**	**Khác**
UCS	☐	☒	☐	☐
Greedy h_1	☐	☐	☐	☒ A-B-E-G (hoặc A-B-D-E-G)
Greedy h_2	☒	☐	☐	☐
A* h_1	☐	☐	☒	☐
A* h_2	☐	☒	☐	☐

b) Chi phí tối ưu bằng UCS = 11, theo đường A-C-E-F-G (4+2+3+2).

c) h_1:

Admissible: Không
Consistent: Không

d) h_2:
Admissible: Có
Consistent: Không

Câu 4:
a)
Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.
Sai
Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.
Đúng
Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.
Đúng
Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.
Sai
Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.
Sai
Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.
Đúng

b)

Trạng thái	h(s)
S	4
A	1
C	2
G	0